IPA 1/2569

Plan/Apply Lifecycle & Understanding State

plan

apply

state

DAG

WEEK 14

AWS Academy Learner Lab — Environment & Constraints

WEEK 14

Read before starting Week 14 lab — these rules are enforced by the Lab policy

Regions	us-east-1 or us-west-2 only All resources must be created in one of these two regions.
Credentials	Provided by the Lab panel — click "AWS Details" → copy Access Key/Secret Key. Export as environment variables or use the credential file. NEVER hardcode in .tf files.
EC2 Instance Types	nano, micro, small, medium, large only. t3.micro recommended. xlarge and above are automatically terminated by the Lab monitor.
IAM — CRITICAL	Cannot create new IAM roles. Use pre-existing LabRole and LabInstanceProfile. PassRole is allowed only for specific pre-defined roles.
EBS Volumes	Types: gp2, gp3, sc1, standard only (no io1/io2). Max size: 100 GB. Enforced by policy.
No Spot/Reserved	Spot, Reserved, and Scheduled instances are denied. Only on-demand, default tenancy.

Week 14 — Specific Lab Notes

State file location

Local state file is fine for this week's lab — remote backend is Week 15.
Do NOT commit terraform.tfstate to git — add to .gitignore.

Using -target

-target is available and works. Useful for isolating specific resources.
Warning: may miss hidden dependencies — use carefully.

Region in plan output

All resources must target us-east-1 or us-west-2.
If plan shows wrong region, check provider block and TF_VAR_aws_region.

Spot / Reserved in plan

If you accidentally set spot = true, the policy will deny apply.
Stick to on-demand instances (default behaviour).

Week 14 Agenda

WEEK 14

3 hours: ~60 min concept + ~120 min hands-on lab

- 5.1 — Directed acyclic graphs (DAGs) — why dependency order matters
- 5.2 — Terraform resource graph — nodes, terraform graph command, modules
- 5.3 — Plan modes — -replace, -target, refresh, plan files, terraform show
- 5.4 — Root-level input variables — -var, .tfvars, env vars, precedence, secrets
- 5.5 — Apply — plan+apply together, saved plan files, destroy
- 5.6 — Apply/plan options — parallelism, locking, JSON output
- 5.7 — Common pitfalls — cycles, cascades, hidden deps, always-detected changes
- 6.1 — Purpose of state — real-world linkage, performance, state-only resources
- 6.2 — State considerations — resiliency, security, availability
- 6.3 — Dissecting state — JSON, lineage, serial, resources, outputs
- **LAB: Inspect dependency graph, use plan files, explore state JSON**

Dependency Graphs (DAGs)

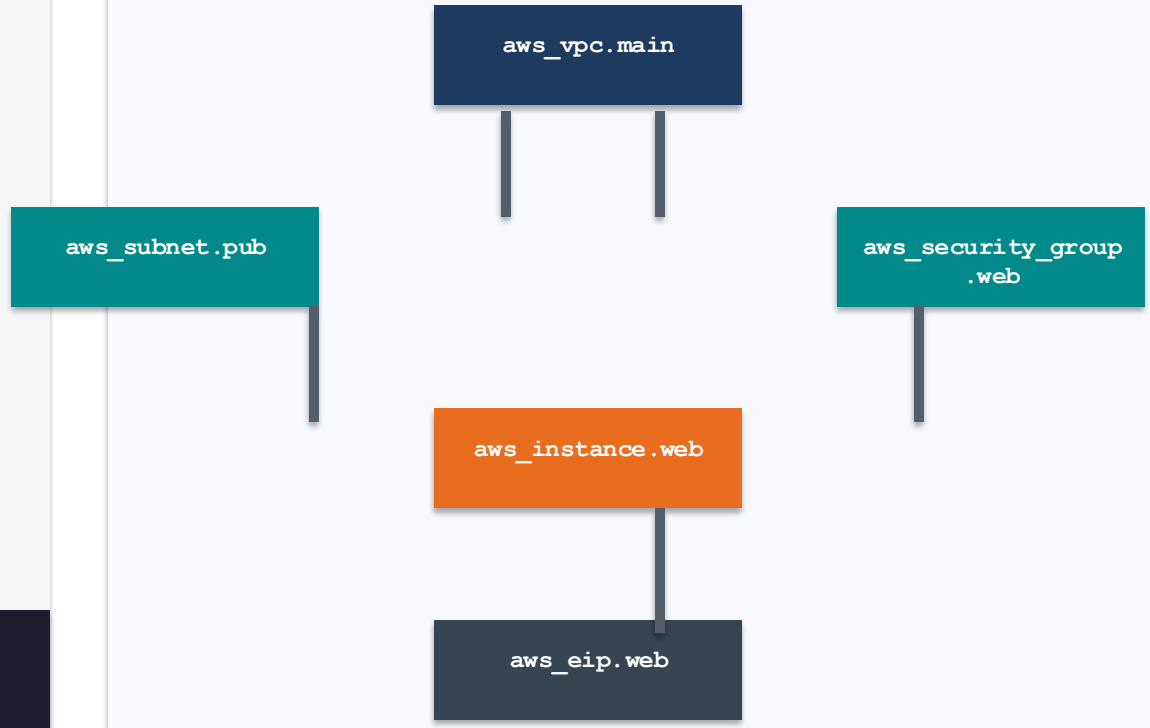
How Terraform determines the order of operations | Ch. 5.1–5.2

Directed Acyclic Graph (DAG)

- Directed: edges point from dependency → dependent
- Acyclic: no circular references allowed
- Terraform builds the graph automatically from references
- Resources with no dependency run in parallel
- terraform graph outputs Graphviz DOT format
- Paste DOT output at dreampuf.github.io/GraphvizOnline

```
# Generate dependency graph
terraform graph | dot -Tsvg > graph.svg
terraform graph -type=plan > plan.dot
terraform graph -draw-cycles
```

Example: EC2 with VPC + SG



Parallel: vpc → subnet and vpc → sg run at the same time

Sequential: instance waits for BOTH subnet and sg to finish

Plan Modes & Options

Controlling what Terraform proposes | Ch. 5.3

Planning Modes

Normal (default)

Compares desired state (config) with current state. Proposes changes to reconcile.

Destroy mode

terraform plan -destroy
Proposes destroying all managed resources. Preview before destroy.

Refresh-only

terraform plan -refresh-only
Updates state to match real-world without changing anything.

```
terraform plan -out=tfplan.bin
terraform apply tfplan.bin
```

Always apply the exact plan you reviewed

Targeting & Replacement Flags

`-target=resource.name`

Limit plan/apply to specific resource. Useful for debugging.
Warning: may miss hidden dependencies.

`-replace=resource.name`

Force recreation of a resource (replaces deprecated taint).
terraform plan -replace=aws_instance.web

`-refresh=false`

Skip querying real-world state. Faster but may miss drift.

`-parallelism=N`

Default: 10 concurrent operations. Reduce if hitting API rate limits.

`-lock=false`

Skip state locking. DANGEROUS — only for debugging.

`-compact-warnings`

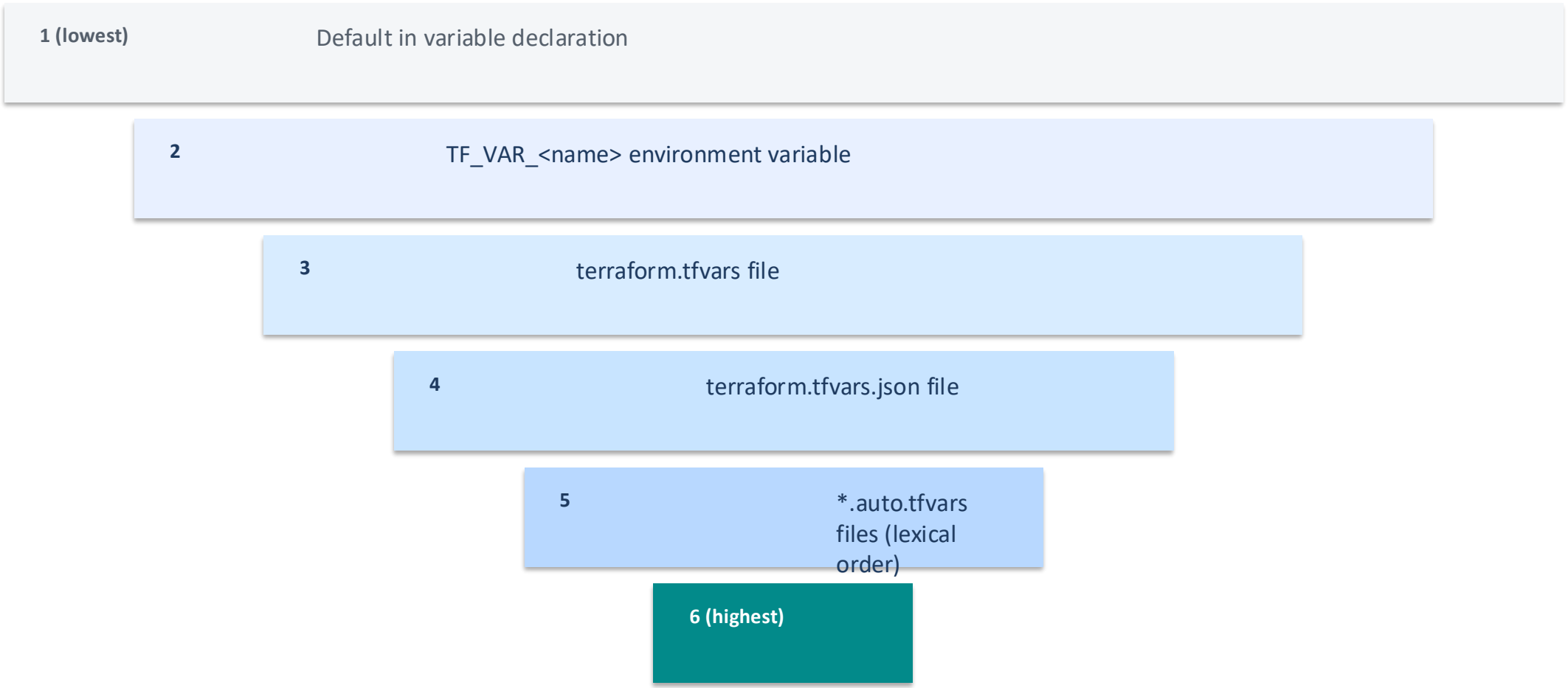
Show warnings in compact form (one line each).

`-json`

Machine-readable JSON output. Useful for CI pipelines.

Root-Level Variable Input

How Terraform reads values — precedence order | Ch. 5.4




```
# Ways to pass variables (shown highest → lowest priority):
terraform plan -var="env=prod" -var-file=prod.tfvars  # CLI flags
export TF_VAR_env=prod && terraform plan   # env var: beaten by any .tfvars
```

Common Pitfalls & Errors

What goes wrong — and how to fix it | Ch. 5.7

Circular dependency

Problem: Resource A depends on B which depends on A.
Terraform reports: "Cycle: resource.A, resource.B"

Fix: Remove the cycle. Use depends_on as last resort.
Often caused by referencing the wrong attribute.

Hidden dependency

Problem: Resource B needs A to exist first, but no HCL reference links them — Terraform runs them in parallel.

Fix: Add depends_on = [resource.a] explicitly to force ordering.

Calculated values & iteration

Problem: for_each or count uses a value known only after apply — Terraform refuses to plan.

Fix: Pre-compute the value in a local or use -target to create the dependency first.

Cascading changes

Problem: Changing one resource triggers unexpected recreation of dependent resources.

Fix: Use ignore_changes lifecycle to skip volatile attrs.
Or use -target to isolate. Review plan carefully.

Always-detected change

Problem: terraform plan always shows a change even after apply. Provider constantly reports drift.

Fix: Add the attribute to ignore_changes. Or check provider docs — some attrs are computed.

Failed state update

Problem: Error mid-apply leaves state inconsistent.
Some resources created, some not.

Fix: Re-run terraform apply — Terraform picks up where it left off. Fix underlying error first.

Purpose of State

Why Terraform needs a state file | Ch. 6.1

Real-World Linkage

- Terraform config describes desired state
- State file maps config → actual resource IDs
- E.g. aws_instance.web ↔ i-0abc123def
- Without state, Terraform can't track what it manages

Reduced Complexity

- Terraform queries only managed resources (not all AWS)
- State cache avoids repeated API calls during plan
- Large environments would be slow without caching
- Only changed resources are refreshed

Performance

- terraform plan skips querying unmanaged resources
- Parallel operations guided by dependency graph
- -refresh=false skips refresh using cached state
- Critical for large deployments (100s of resources)

State-Only Resources

- Some resources exist only in state (no real API)
- random_string, random_id: stable pseudo-random values
- null_resource: lifecycle hooks with triggers
- terraform_data: store arbitrary values in state

Dissecting the State File

What's inside terraform.tfstate | Ch. 6.2–6.3

```
{
  "version": 4,
  "terraform_version": "1.16.3",
  "serial": 12,          // increments on every apply
  "lineage": "f3a4....", // unique UUID for this state
  "outputs": {
    "instance_id": {
      "value": "i-0abc123",
      "type": "string"
    }
  },
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "provider": "provider[\"registry.../aws\"]",
      "instances": [{
        "schema_version": 1,
        "attributes": {
          "id": "i-0abc123def",
          "ami": "ami-0abc...",
          "instance_type": "t3.micro"
        }
      }]
    }
  ]
}
```

version

State schema version. Increments on major Terraform releases.

serial

Monotonically increasing counter. Increments on every apply. Used for conflict detection.

lineage

UUID generated on terraform init. Identifies this state's "family". Mismatched lineage = error.

resources[]

Array of every managed resource. Each has type, name, provider, and instances[].

attributes

Complete snapshot of every attribute Terraform knows about the resource.

outputs

Root module outputs stored for cross-reference and terraform output.

NEVER hand-edit

Edit state via CLI commands only (state mv, state rm, import). Manual edits corrupt lineage.

Sensitive data

Sensitive values are stored in plaintext. Secure state storage is critical (Week 15).

HANDS-ON LAB

Week 14 Lab — Inspect Lifecycle, Plan Files & State

Objective: Drive and inspect the plan/apply lifecycle; look inside the state file

1

terraform graph

Generate dependency graph
Pipe to DOT / SVG
Identify resource ordering

2

Saved plan file

terraform plan -out=plan.bin
terraform show plan.bin
terraform apply plan.bin

3

-target & -replace

Plan with -target=module.web_server
Use -replace to force recreation
Observe resulting plan diff

4

Variable precedence

Supply via .tfvars, -var flag
Set a TF_VAR_ env variable
Verify precedence order

5

Inspect state file

Open terraform.tfstate (JSON)
Find lineage, serial, attributes
Do NOT hand-edit!

6

Reproduce a pitfall

Introduce a circular dependency
Read the error message
Fix it and re-apply

Deliverable: Saved plan file + written observations on state structure + resolved pitfall

Week 14 Lab — Plan/Apply Lifecycle & State (AWS Academy)

Objective: Inspect dependency graph, use plan files, explore state — all fully allowed in Learner Lab

1

terraform graph

Generate dependency graph
Pipe to DOT / SVG
Identify resource ordering

2

Saved plan file

terraform plan -out=plan.bin
terraform show plan.bin
terraform apply plan.bin

3

-target & -replace

Plan with -target=module.web_server
Use -replace to force recreate
Observe resulting plan diff

4

Variable precedence

Supply via .tfvars, -var flag
Set a TF_VAR_ env variable
Verify precedence order

5

Inspect state

Open terraform.tfstate (JSON)
Find lineage, serial, attributes
Do NOT hand-edit

6

Reproduce pitfall

Introduce a circular dep
Read the error message
Fix it and re-apply

Week 14 — Key Takeaways

WEEK 14

What you should be able to do after today

Dependency Graphs

Terraform auto-resolves order from HCL references. terraform graph shows it. Parallel = faster.

Plan Modes

Normal / destroy / refresh-only. Save with -out, apply saved plan. -target and -replace for surgical ops.

Variable Precedence

CLI -var/-var-file > auto.tfvars > terraform.tfvars > TF_VAR_ env > default. Env vars rank LOW.

Common Pitfalls

Cycles, cascading changes, always-detected diffs, hidden deps. Each has a specific fix — know them.

Purpose of State

Maps config → real resource IDs. Cache for performance. lineage + serial for conflict detection.

State is Sensitive

Plaintext JSON including secrets. Never hand-edit. Secure it — Week 15 moves it to S3.

Next week: Remote state (S3 backend), state management CLI, drift, code quality & security → Ch. 6.4–6.8 + 7.3–7.5